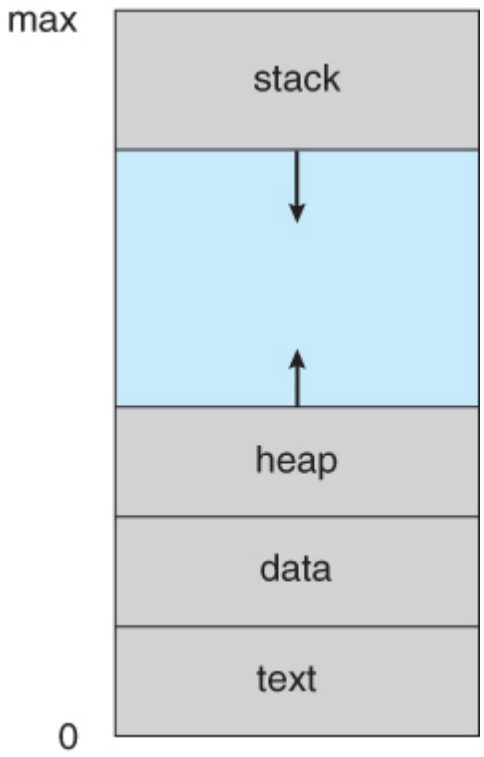
PROCESS MANAGEMENT

PROCESS CONCEPT

- A process is an instance of a program in execution.
- Batch systems work in terms of "jobs". Many modern process concepts are still expressed in terms of jobs, (e.g. job scheduling), and the two terms are often used interchangeably.

THE PROCESS

- Process memory is divided into four sections:
    - The **text section** comprises the compiled program code, read in from non-volatile storage when the program is launched.
    - The **data section** stores global and static variables, allocated and initialized prior to executing main.
    - The **heap** is used for dynamic memory allocation, and is managed via calls to new, delete, free, etc.
    - The **stack** is used for local variables. Space on the stack is reserved for local variables when they are declared (at function entrance or elsewhere, depending on the language), and the space is freed up when the variables go out of scope. Note that the stack is also used for function return values, and the exact mechanisms of stack management may be language specific.
    - Note that the stack and the heap start at opposite ends of the process's free space and grow towards each other. If they should ever meet, then either a stack overflow error will occur, or else a call to new or malloc will fail due to insufficient memory available.
- When processes are swapped out of memory and later restored, additional information must also be stored and restored. Key among them are the program counter and the value of all program registers.



A process in memory

PROCESS CREATION

When a program is executed, process is created. Process creation is achieved through the fork() system call. Four principal event causes process to be created;

a. System initialization

b. Execution of a process-creation system call by a running process.

c. A user request to create a new process

d. Initiation of a batch job

Note: System call is a mechanism that provides the interface between a process and the operating system. It is also a function that a user program uses to ask the operating system for a particular service. When a process in user mode needs access to a resource, system calls are usually generated.

PROCESS TERMINATION

After a process has been created, it starts running and does whatever its job is. However, nothing lasts forever, not even processes. Sooner or later, the new process will terminate, usually due to one of the following conditions;

1. **Normal exit (voluntary):** most processes terminate because they have done their work. When a compiler has complied the program given to it, the compiler executes a system call to tell the operating system that it is finished' this call is exit in UNIX and ExitProcess in windows.

2. **Error exit (voluntary):** the second reason for termination is that the process discovers a fatal error eg. If a user types the command cc foo.c to compile the program foo.c and no such file exists, the compiler simply announces this fact and exists.

3. **Fatal error (involuntary):** reason for this termination is an error caused by the process often due to a program bug e.g, executing an illegal instruction, dividing by zero.

4. **Killed by another process (involuntary):** reason a process might terminate is that the process executes a system call telling the operating system to kill some other process. In UNIX, this call KILL, the corresponding win32 function is TerminateProcess.

In some systems, when a process terminates, either voluntarily or otherwise, all processes it created are immediately killed as well.

PROCESS STATE

As process executes, it changes state. The state of a process is defined in part by the current activity of that process. Processes may be in one of following states as shown in Figure 3.2 below.

- **New** - The process is in the stage of being created.

- **Ready** - The process has all the resources available that it needs to run, but the CPU is not currently working on this process's instructions. The process may enter this state after starting or while running, but the scheduler may interrupt it to assign the cpu to another process.

- **Running** - The CPU is working on this process's instructions.

- **Waiting** - The process cannot run at the moment, because it is waiting for some resource to become available or for some event to occur. For example the process may be waiting for keyboard input, disk access request, inter-process messages, a timer to go off, or a child process to finish.

- **Terminated** - The process has finished execution and is relocated to the terminated state where it waits for removal from the main memory once it has

completed its execution or been terminated by the operating system.

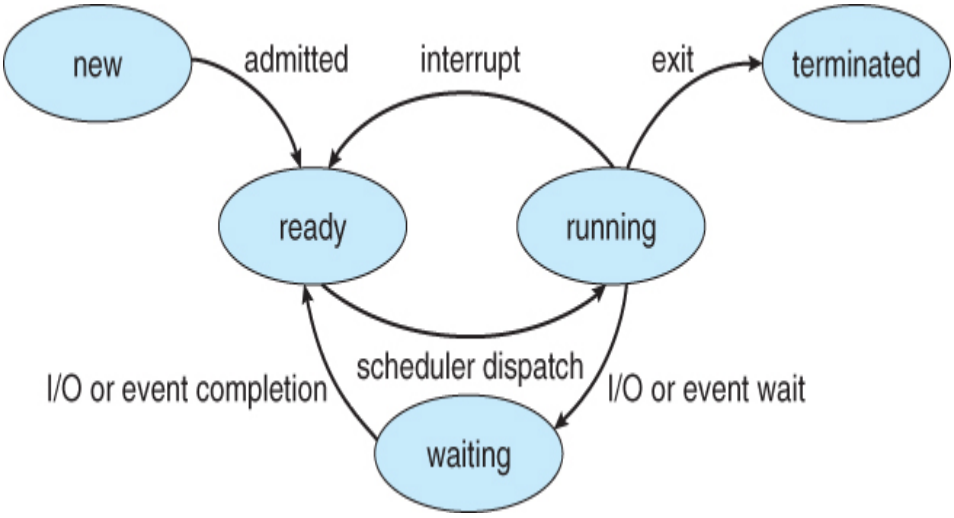


Diagram of process state

PROCESS CONTROL BLOCK (PCB)

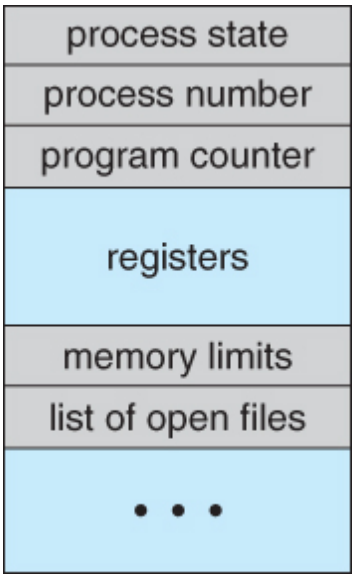
Every process has a process control block, which is a data structure managed by the operating system. An integer process ID (or PID) is used to identify the PCB.

PCB stores all the information required to maintain track of a process, these includes;

- **Process State**: The process's present state such as whether its running, waiting, etc., as discussed above.
- **Process ID**, and parent process ID.
- **Process Privileges:** This is required in order to grant or deny access to system resources.
- **CPU registers:** processes must be stored in various CPU registers for execution in the running state. The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers and general-purpose registers etc.
- **Program Counter** - These need to be saved and restored when swapping processes in and out of the CPU.

- **CPU-Scheduling information** - Such as priority information and pointers to scheduling queues.
- **Memory-Management information** – This includes information from the page tables or segment tables, memory limitations etc all of which are dependent on the amount of memory used by the operating system.
- **Accounting information** - user and kernel CPU time consumed, account numbers, limits, etc.
- **I/O Status information** - Devices allocated, open file tables, etc.

The PCB architecture is fully dependent on the OS and different OS may include different information.



Process control block (PCB)

OPERATIONS ON PROCESSES

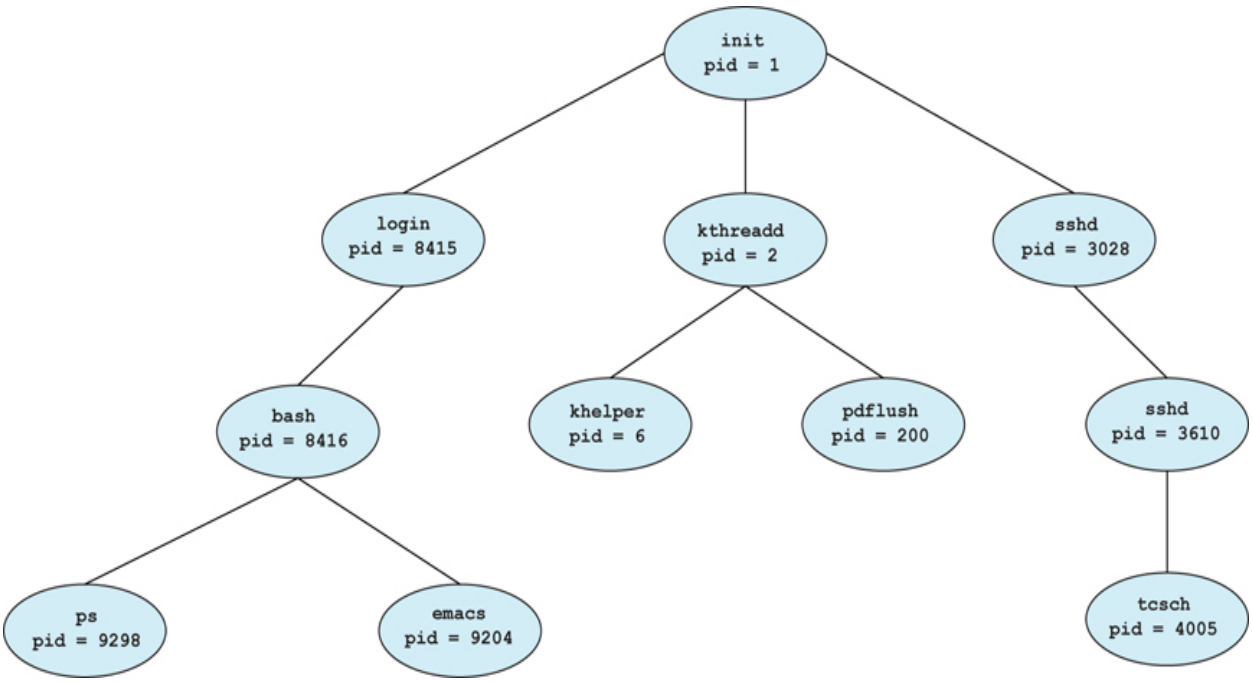
The user can perform the following operations on a process in the operating system:

a. Process Creation
b. Process scheduling or dispatching
c. Blocking
d. Preemption
e. Termination

Process Creation

Process creation is the initial step to process execution. It implies the creation of a new process for execution.

- Processes may create other processes through appropriate system calls, such as **fork** or **spawn**. The process which does the creating is termed the **parent** of the other process, which is termed its **child**.
- Each process is given an integer identifier, termed its **process identifier**, or PID. The parent PID (PPID) is also stored for each process.
- On typical UNIX systems the process scheduler is termed **sched**, and is given PID 0. The first thing it does at system startup time is to launch **init**, which gives that process PID 1. Init then launches all system daemons and user logins, and becomes the ultimate parent of all other processes. The figure below shows a typical process tree for a Linux system, and other systems will have similar though not identical trees:



A tree of processes on a typical Linux system

- Depending on system implementation, a child process may receive some amount of shared resources with its parent. Child processes may or may not be limited to a subset of the resources originally allocated to the parent, preventing runaway children from consuming all of a certain system resource.

- There are two options for the parent process after creating the child:
    1. Wait for the child process to terminate before proceeding. The parent makes a wait( ) system call, for either a specific child or for any child, which causes the parent process to block until the wait( ) returns. UNIX shells normally wait for their children to complete before issuing a new prompt.
    2. Run concurrently with the child, continuing to process without waiting. This is the operation seen when a UNIX shell runs a process as a background task. It is also possible for the parent to run for a while, and then wait for the child later, which might occur in a sort of a parallel processing operation. (E.g. the parent may fork off a number of children without waiting for any of them, then do a little work of its own, and then wait for the children.)

Process Scheduling

Scheduling or dispatching refers to the event where the OS puts the process from the ready state to the running state. It is done by the system when there are free resources or there is a process of higher priority than the ongoing process.

Blocking

Block mode is a mode where the system waits for input-output. In process blocking operation, the system puts the process in the waiting state. When a task is blocked, it is unable to execute until the task prior to it has finished using the shared resource, example of shared resources are the CPU, network and network interfaces, memory and disk.

Preemption

Preemption means the ability of the operating system to preempt a currently scheduled task in favour of a higher priority task. The resource being scheduled can be the processor or the I/O

Process Termination

- Processes may request their own termination by making the **exit( )** system call, typically returning an int. This int is passed along to the parent if it is doing a **wait( )**, and is typically zero on successful completion and some non-zero code in the event of problems.
- Processes may also be terminated by the system for a variety of reasons, including:

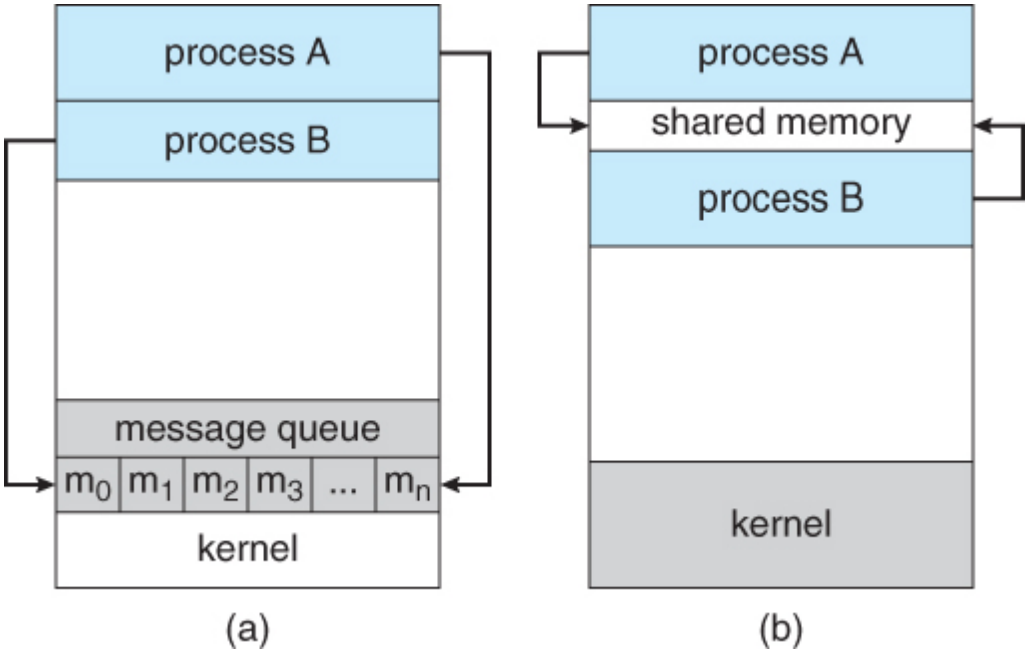
    - The inability of the system to deliver necessary system resources.
    - In response to a KILL command, or other unhandled process interrupt.
    - A parent may kill its children if the task assigned to them is no longer needed.
    - If the parent exits, the system may or may not allow the child to continue without a parent.
- When a process terminates, all of its system resources are freed up, open files flushed and closed, etc. The process termination status and execution times are returned to the parent if the parent is waiting for the child to terminate, or eventually returned to init if the process becomes an orphan. (Processes which are trying to terminate but which cannot because their parent is not waiting for them are termed **zombies**. These are eventually inherited by init as orphans and killed off.

INTER-PROCESS COMMUNICATION

Processes executing concurrently in the operating system may be either independent process or cooperating processes

- **Independent Processes** operating concurrently on a system are those that can neither affect other processes or be affected by other processes.
- **Cooperating Processes** are those that can affect or be affected by other processes. Clearly, any process that shares data with other processes is a cooperating process. There are several reasons why cooperating processes are allowed:
    - Information Sharing - There may be several processes which need access to the same file for example. (e.g. pipelines.)
    - Computation speedup - Often a solution to a problem can be solved faster if the problem can be broken down into sub-tasks to be solved simultaneously (particularly when multiple processors are involved.)
    - Modularity - The most efficient architecture may be to break a system down into cooperating modules. (E.g. databases with a client-server architecture.)
    - Convenience - Even a single user may be multi-tasking, such as editing, compiling, printing, and running the same code in different windows.

Cooperating processes require some type of inter-process communication, which is most commonly one of two types: Shared Memory systems or Message Passing systems. The figure below illustrates the difference between the two systems:



Communications models: (a) Message passing. (b) Shared memory.

- Shared Memory is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. However it is more complicated to set up, and doesn't work as well across multiple computers. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.
- Message Passing requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small, or when multiple computers are involved.